



IITMO

Gen CV: Basic

Современные архитектуры нейронных сетей

Применение Gen CV



Image generation
(Midjourney v7)



Video generation
(Sora 2)

Что такое распределение?

В статистике и машинном обучении **распределение вероятностей (distribution)** описывает, насколько вероятны разные значения случайной величины x .

Пример: если x — рост человека, распределение показывает, с какой вероятностью встретится человек ростом 160, 170 или 180 см.

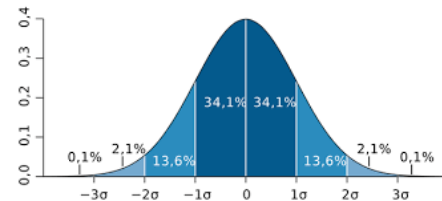
Формально: распределение описывается **функцией плотности вероятности** $p(x)$, где

$$\int p(x) dx = 1.$$

Примеры распределений

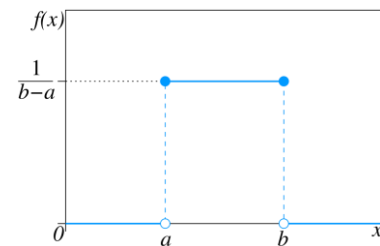
1 Нормальное распределение (Gaussian)

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



2 Равномерное распределение (Uniform)

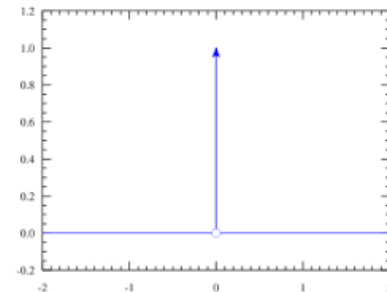
$$p(x) = \begin{cases} \frac{1}{b-a}, & a \leq x \leq b \\ 0, & \text{иначе} \end{cases}$$



3 Дельта-распределение Дирака (Dirac delta distribution)

$$p(x) = \delta(x - x_0)$$

- $\delta(x) = \begin{cases} +\infty, & x = 0, \\ 0, & x \neq 0; \end{cases}$
- $\int_{-\infty}^{+\infty} \delta(x) dx = 1.$



Важные формулы 1

Law of the unconscious statistician (LOTUS)

Пусть x — случайная величина с плотностью $p_x(x)$, и пусть $y = f(x)$ с плотностью $p_y(y)$. Тогда

$$\mathbb{E}_{p_y} y = \int p_y(y) y dy = \int p_x(x) f(x) dx = \mathbb{E}_{p_x} f(x).$$

Закон неосознанного статистика (LOTUS) говорит, что

матожидание функции от случайной величины можно считать по распределению самой случайной величины, даже если распределение функции $y = f(x)$ неизвестно.

Monte-Carlo estimation

Математическое ожидание можно оценить, используя только выборку:

$$\mathbb{E}_{p(x)} f(x) = \int p(x) f(x) dx \approx \frac{1}{n} \sum_{i=1}^n f(x_i) \quad , \text{ где } x_i \sim p(x).$$

Метод Монте-Карло позволяет приближённо вычислить интегралы, используя случайные выборки из распределения $p(x)$.

Важные формулы 2

Jensen's Inequality

Пусть x — случайная величина, а $f(\cdot)$ — **выпуклая функция**

(то есть функция, для которой выполняется условие

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

для всех x_1, x_2 и $\lambda \in [0, 1]$.)

Тогда выполняется неравенство:

$$\mathbb{E}[f(x)] \geq f(\mathbb{E}[x]).$$

Интуитивно: **функция от среднего $\leq$ среднее от функции**, если функция выпуклая.

Decomposition to conditionals

Пусть $\mathbf{x} = (x_1, x_2, \dots, x_m) \in \mathbb{R}^m$ — случайная величина. Тогда:

$$p(\mathbf{x}) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_2, x_1) \cdots p(x_m | x_{m-1} \dots x_1).$$

Это правило разложения вероятности:

сложное распределение можно выразить через произведение условных вероятностей.

Постановка задачи генеративного моделирования



Нам даны независимые и одинаково распределённые выборки $\{x_i\}_{i=1}^n \in \mathcal{X}$ (например, $\mathcal{X} = \mathbb{R}^m$) из неизвестного распределения $\pi(x)$.

Goal

Мы хотим обучить распределение $\pi(x)$ для:

- оценки $\pi(x)$ для новых выборок (насколько вероятно получить объект x);
- выборки из $\pi(x)$ (чтобы получить новые объекты $x \sim \pi(x)$).

Challenge

- Данные являются сложными и высокоразмерными.
- Например, набор изображений лежит в пространстве $\mathcal{X} \subset \mathbb{R}^{(\text{width} \times \text{height} \times \text{channels})}$.

Постановка задачи генеративного моделирования



Зафиксируем вероятностную модель $p(x | \theta)$ — множество параметризованных распределений. Вместо того чтобы искать истинное распределение $\pi(x)$ среди всех возможных вероятностных распределений, обучаем аппроксимацию функции $p(x | \theta) \approx \pi(x)$.

What is a divergence?

Пусть $\mathcal{S}$ — множество всех возможных вероятностных распределений.

Тогда $D: \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$ называется **divergence**, если выполняются условия:

- $D(\pi \parallel p) \geq 0$ для всех $\pi, p \in \mathcal{S}$;
- $D(\pi \parallel p) = 0$ тогда и только тогда, когда $\pi \equiv p$.

Divergence minimization task

где $\pi(x)$ — истинное распределение данных,
а $p(x | \theta)$ — распределение, задаваемое моделью.

$$\min_{\theta} D(\pi \parallel p),$$

Kullback–Leibler divergence

$$D_{\text{KL}}(p(x) \parallel q(x)) = \int_{-\infty}^{\infty} p(x) \ln \frac{p(x)}{q(x)} dx$$

Forward KL vs Reverse KL



Forward KL

$$KL(\pi \parallel p) = \int \pi(x) \log \frac{\pi(x)}{p(x | \theta)} dx \rightarrow \min_{\theta}$$

Reverse KL

$$KL(p \parallel \pi) = \int p(x | \theta) \log \frac{p(x | \theta)}{\pi(x)} dx \rightarrow \min_{\theta}$$

Maximum likelihood estimation (MLE)

Пусть $\mathbf{X} = \{x_i\}_{i=1}^n$ — множество независимых выборов.

Тогда

$$\theta^* = \arg \max_{\theta} p(\mathbf{X} | \theta) = \arg \max_{\theta} \prod_{i=1}^n p(x_i | \theta) = \arg \max_{\theta} \sum_{i=1}^n \log p(x_i | \theta).$$

Forward KL vs Reverse KL



Forward KL

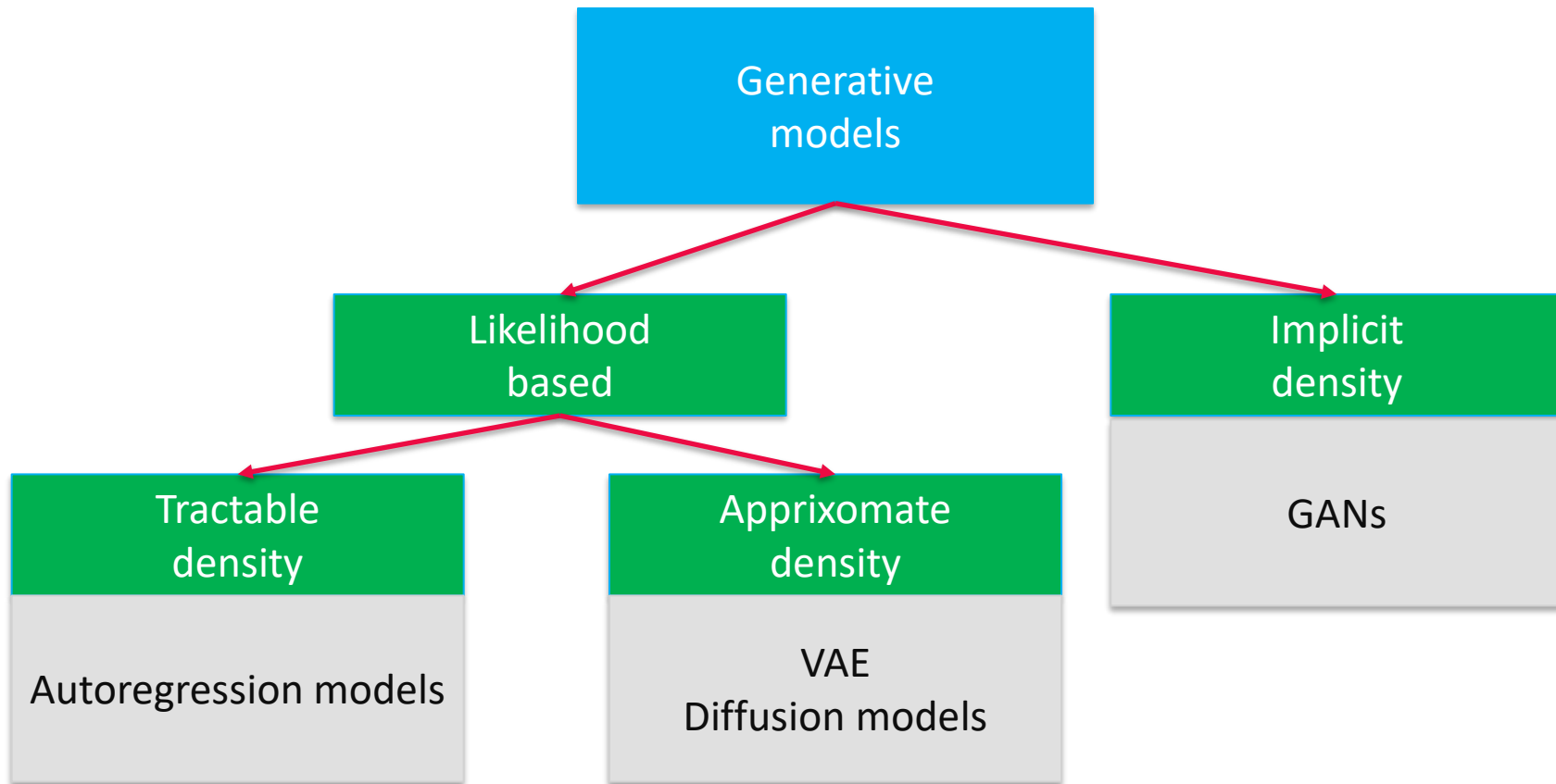
$$\begin{aligned} KL(\pi \parallel p) &= \int \pi(x) \log \frac{\pi(x)}{p(x | \theta)} dx \\ &= \int \pi(x) \log \pi(x) dx - \int \pi(x) \log p(x | \theta) dx \\ &= -\mathbb{E}_{\pi(x)}[\log p(x | \theta)] + \text{const} \\ &\approx -\frac{1}{n} \sum_{i=1}^n \log p(x_i | \theta) + \text{const} \rightarrow \min_{\theta} \end{aligned}$$

Maximum likelihood estimation (MLE) эквивалентна минимизации оценки Монте-Карло для **forward KL**.

Reverse KL

$$\begin{aligned} KL(p \parallel \pi) &= \int p(x | \theta) \log \frac{p(x | \theta)}{\pi(x)} dx \\ &= \mathbb{E}_{p(x|\theta)}[\log p(x | \theta) - \log \pi(x)] \rightarrow \min_{\theta} \end{aligned}$$

Зоопарк генеративных моделей



Autoregression models (ARMs)

Goal

Смоделировать распределение $\pi(\mathbf{x})$ для изображений из реального мира.

Solution

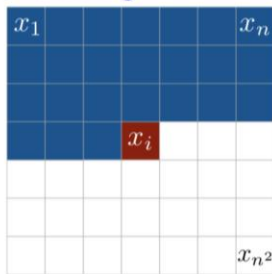
Авторегрессионная модель по 2D-пикселям:

$$p(\mathbf{x} \mid \theta) = \prod_{j=1}^{width \times height} p(x_j \mid \mathbf{x}_{1:j-1}, \theta)$$

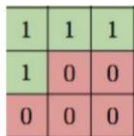
- 1) Необходимо ввести **порядок обхода пикселей** (raster ordering).
- 2) Свёртку нужно **маскировать**, чтобы сделать её **каузальной** (не "подглядывала" в будущее пиксели).
- 3) Поскольку изображение имеет **RGB-каналы**, зависимости между ними также должны учитываться.

PixelCNN

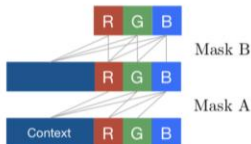
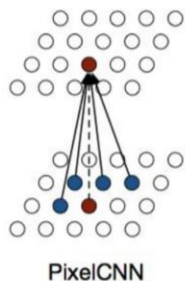
Raster ordering



Mask for the convolution kernel



Dependencies between pixels



- **Input:** изображение (например, $32 \times 32 \times 3$).
- **Layer 1:** *Masked Conv2D (Mask A)* — учитывает только предыдущие пиксели.
- **Next layers:** *Masked Conv2D (Mask B)* — используют текущий пиксель и прошлый контекст.
- **Activations:** *ReLU*
- **Output:** вероятности для всех возможных значений пикселя (softmax)
→ *Categorical distribution* (обычно 256 классов).

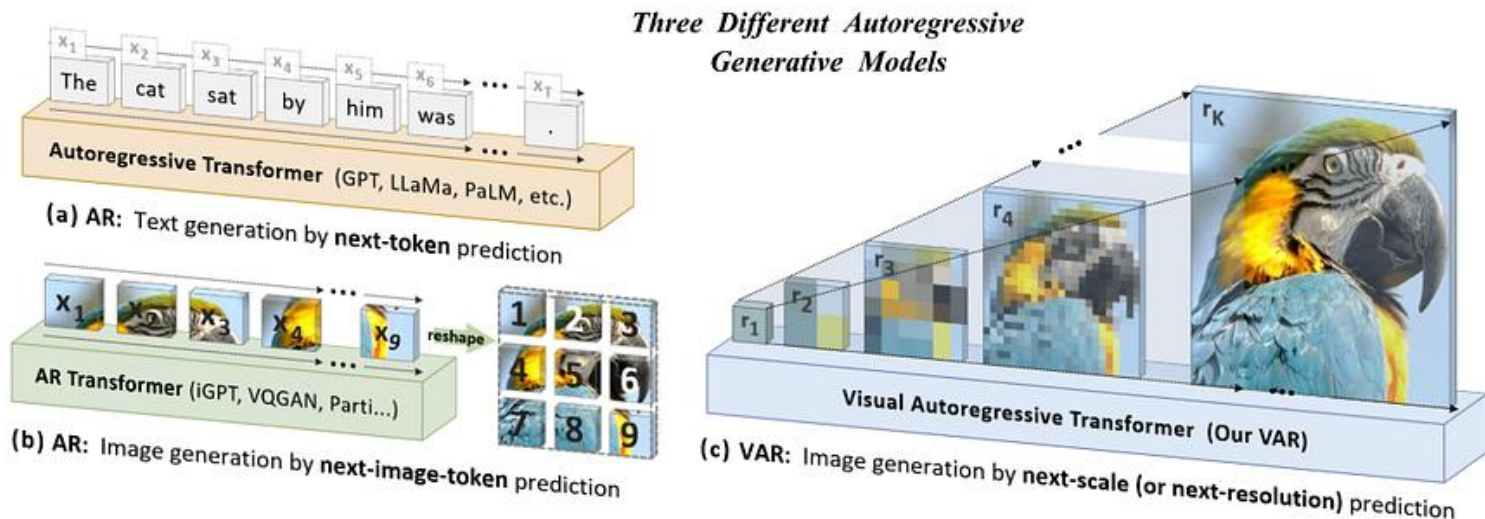


Figure 2: Standard autoregressive modeling (AR) vs. our proposed visual autoregressive modeling (VAR). (a) AR applied to language: sequential text token generation from left to right, word by word; (b) AR applied to images: sequential visual token generation in a raster-scan order, from left to right, top to bottom; (c) VAR for images: multi-scale token maps are autoregressively generated from coarse to fine scales (lower to higher resolutions), with parallel token generation within each scale. VAR requires a multi-scale VQVAE to work.

Плюсы и минусы ARMs

Тип	оценка $\pi(x)$	Генерация из $\pi(x)$
ARMs	Да Для PixelCNN – 1 inference step	Да Для PixelCNN – $h * w$ inference steps

Generative adversarial networks (GANs)

Likelihood-based models: quality vs likelihood



Проблема

Высокое **log-likelihood** не гарантирует хорошее качество сэмплов, а хорошие сэмплы не означают высокий **likelihood**.

Пример 1 — *Poor likelihood, great samples*

$$p_1(x) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(x, |x_i, \epsilon)$$

При очень малом ϵ каждая компонента становится узкой, и сэмплы выглядят **реалистично** (почти копии обучающих данных).

Но тестовые примеры имеют **очень низкое правдоподобие** — модель переобучается на обучающие точки.

!Качество высокое, но модель не обобщает.

Likelihood-based models: quality vs likelihood



Пример 2 — *Great likelihood, poor samples*

$$p_2(x) = 0.01p(x) + 0.99p_{noise}(x)$$

Модель “разбавляет” реальное распределение шумом. Это повышает средний log-likelihood:

$$\log[0.01p(x)+0.99p_{noise}(x)] \geq \log[0.01p(x)] = \log[p(x)] - \log 100$$

Но теперь **большинство сэмплов — шум**, несмотря на формально улучшенную метрику.

Вывод

Likelihood — не всегда надёжная мера качества для генеративных моделей.

Модель может иметь:

- отличные сэмплы, но низкий likelihood;
- высокий likelihood, но нерелевантные (мусорные) сэмплы.

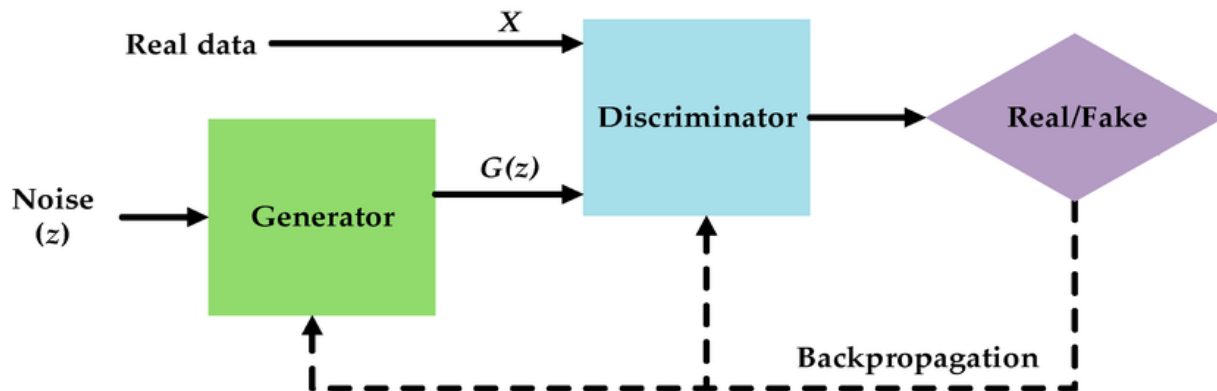
Generator

Генеративная модель $\mathbf{x} = G(\mathbf{z})$, которая делает сгенерированные примеры более реалистичными.

Здесь $\mathbf{z}$ — это выборка из базового (известного) распределения $p(\mathbf{z})$, и $\mathbf{x} \sim p(\mathbf{x} | \theta)$. Например, номарльного

◆ Discriminator

Классификатор $p(y = 1 | \mathbf{x}) = D(\mathbf{x}) \in [0, 1]$, который отличает реальные примеры от сгенерированных.



$$\min_G \max_D \left[\mathbb{E}_{\pi(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{x}|\theta)} \log(1 - D(\mathbf{x})) \right]$$

или эквивалентно:

$$\min_G \max_D \left[\mathbb{E}_{\pi(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(G(\mathbf{z}))) \right]$$

Пояснения:

$D(\mathbf{x})$ — **дискриминатор**, оценивает вероятность того, что пример $\mathbf{x}$ — **реальный**.

$G(\mathbf{z})$ — **генератор**, создаёт сэмплы, чтобы **обмануть дискриминатор**.

$\pi(\mathbf{x})$ — распределение **реальных данных**.

$p(\mathbf{z})$ — распределение **шума (латентных переменных)**.

Смысл оптимизации:

Дискриминатор (D) стремится **максимизировать** вероятность правильно различать реальные и фейковые данные.

Генератор (G) стремится **минимизировать** ту же функцию — он хочет, чтобы дискриминатор не мог отличить фейк от реального.

Итог:

Это **борьба во время обучения** между G и D: $\text{Generator} \leftrightarrow \text{Discriminator}$

!обучение без явного вычисления $p(\mathbf{x})$ -- **Likelihood free**
но с максимальным приближением к реальному распределению данных.

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

GANs problems

Mode collapse (схлопывание мод)

Генератор производит ограниченное число уникальных образцов. Он находит несколько «успешных» примеров, которые обманывают дискриминатор, и начинает их повторять.



Причина: дисбаланс в обучении — дискриминатор обучается быстрее, чем генератор.

GANs problems

Неустойчивая сходимость

Обучение напоминает игру “кошки-мышки” между G и D. Иногда GAN вообще не сходится, а лишь осциллирует.

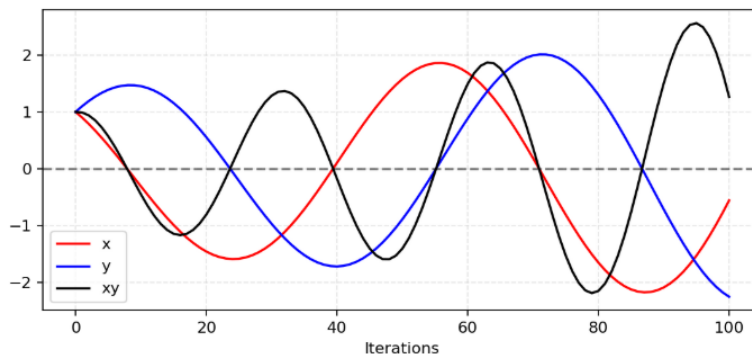


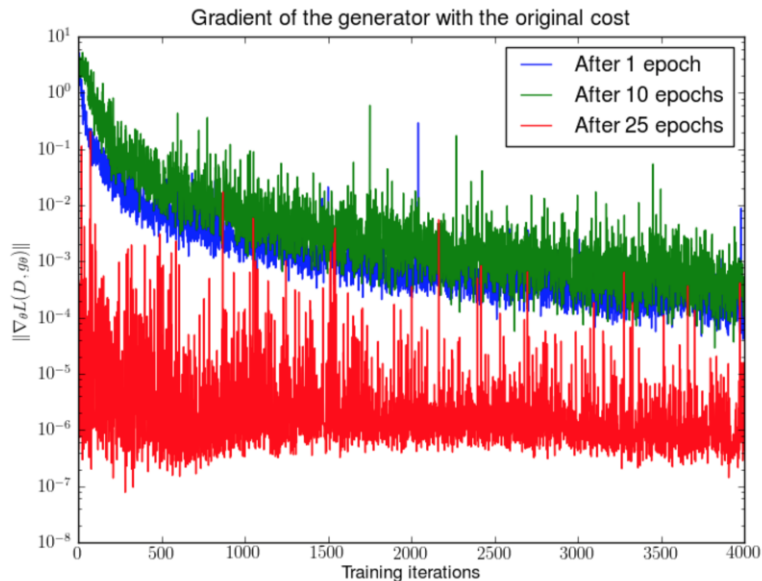
Figure 3: A simulation of our example for updating x to minimize xy and updating y to minimize $-xy$. The learning rate $\eta = 0.1$. With more iterations, the oscillation grows more and more unstable.

Причина: отсутствие единой функции потерь с хорошими градиентами для обеих моделей.

GANs problems

Vanishing gradients

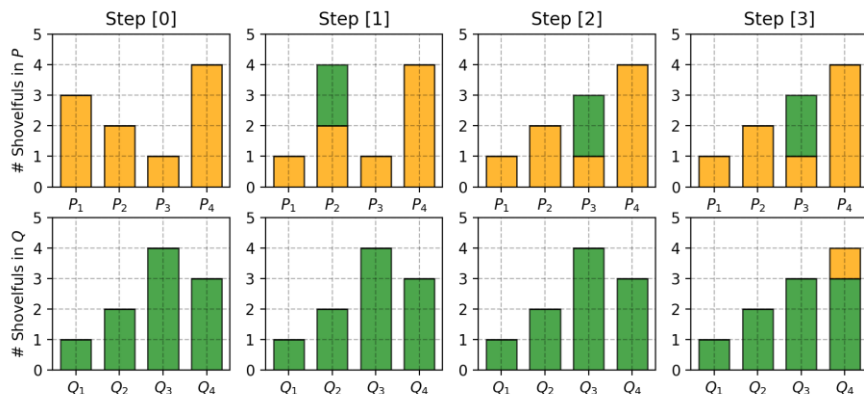
Когда дискриминатор слишком силён, градиенты, поступающие к генератору, исчезают — генератор перестаёт обучаться.



Причина: форма функции потерь в D → насыщение при уверенных предсказаниях.

Wasserstein distance (дискретный случай)

Earth Mover's Distance (EMD) -- Минимальная «стоимость» перемещения и трансформации кучи «грунта» из формы одного распределения вероятностей в форму другого.



$$W(P, Q) = 2(\text{step 1}) + 2(\text{step 2}) + 1(\text{step 3}) = 5$$

Интуиция:

- 1) Представим, что распределение P — это куча песка, а распределение Q — форма, в которую нужно этот песок переместить.
- 2) Wasserstein distance $W(P, Q)$ измеряет **минимальную работу**, необходимую, чтобы преобразовать одно распределение в другое.
- 3) Чем дальше «масса» переносится, тем выше расстояние.

Wasserstein distance (continuous case)



Wasserstein distance (continuous case)

$$W(\pi, p) = \inf_{\gamma \in \Gamma(\pi, p)} \mathbb{E}_{(x, y) \sim \gamma} \|x - y\| = \inf_{\gamma \in \Gamma(\pi, p)} \int \|x - y\| \gamma(x, y) dx dy$$

Интуиция:

Мы ищем **план транспортировки** $\gamma(x, y)$, который показывает, **сколько массы** нужно перенести из точки x (в распределении π) в точку y (в распределении p).

Термины:

$\gamma(x, y)$ — **план транспортировки** (“сколько земли” переносим).

$\Gamma(\pi, p)$ — множество всех распределений $\gamma(x, y)$, чьи маргиналы совпадают с π и p :

$$\int \gamma(x, y) dx = p(y), \int \gamma(x, y) dy = \pi(x)$$

$\|x - y\|$ — **стоимость переноса единицы массы** из точки x в y .

Смысл:

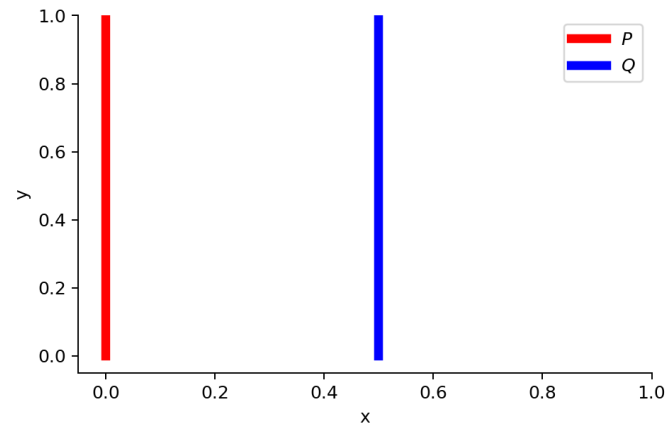
Wasserstein расстояние — это **минимальная средняя работа**, необходимая для преобразования одного распределения в другое.

Оно измеряет не только *насколько* разные распределения, но и *насколько далеко* нужно двигать их массу.

Wasserstein distance VS KL

Suppose we have two probability distributions, P and Q :

$$\forall (x, y) \in P, x = 0 \text{ and } y \sim U(0, 1) \forall (x, y) \in Q, x = \theta, 0 \leq \theta \leq 1 \text{ and } y \sim U(0, 1)$$



When $\theta \neq 0$:

$$D_{KL}(P||Q) = \sum_{x=0, y \sim U(0,1)} 1 \cdot \log \frac{1}{0} = +\infty$$

$$D_{KL}(Q||P) = \sum_{x=\theta, y \sim U(0,1)} 1 \cdot \log \frac{1}{0} = +\infty$$

$$W(P, Q) = |\theta|$$

But when $\theta = 0$, two distributions are fully overlapped:

$$D_{KL}(P||Q) = D_{KL}(Q||P) = D_{JS}(P, Q) = 0$$
$$W(P, Q) = 0 = |\theta|$$

WGAN: использование Wasserstein distance как loss-функции



Проблема:

Прямое вычисление Wasserstein расстояния

$$W(p_r, p_g) = \inf_{\gamma} \int \|x - y\| \gamma(x, y) dx dy$$

невозможно (слишком много возможных планов переноса).

Решение — Канторович–Рубинштейн дуальность:

$$W(p_r, p_g) = \frac{1}{K} \sup_{\|f\|_L \leq K} (\mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)])$$

Вместо «переноса массы» ищем **функцию** f , которая максимально различает реальные и сгенерированные выборки. f должна быть **K-Lipschitz**:

$$|f(x_1) - f(x_2)| \leq K |x_1 - x_2|$$

чтобы гарантировать стабильные градиенты.

Интерпретация:

$f(x)$ — **критик (critic)**, а не бинарный дискриминатор. Он оценивает, *насколько далеки* распределения p_r и p_g .

Чем меньше значение $W(p_r, p_g)$, тем ближе сгенерированные данные к реальным.

WGAN: использование Wasserstein distance как loss-функции



Loss-функция:

$$L(p_r, p_g) = W(p_r, p_g) = \max_{w \in W} (\mathbb{E}_{x \sim p_r}[f_w(x)] - \mathbb{E}_{z \sim p_z}[f_w(g_\theta(z))])$$

f_w — сеть-критик с параметрами w .

$g_\theta(z)$ — генератор, создающий сэмплы из шума z .

Цель:

Критик: максимизирует разницу между реальными и фейковыми.

Генератор: минимизирует эту разницу.

Важный момент — K-Lipschitz условие:

Чтобы f_w оставался Lipschitz, после каждого шага обучения веса ограничивают:

$$w \in [-0.01, 0.01]$$

—это называется **weight clipping**. Это приближённо гарантирует, что f_w не «взрывается» и остаётся 1-Lipschitz.

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size.
 n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w [\frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

Плюсы и минусы GANs

Тип	оценка $\pi(x)$	Генерация из $\pi(x)$
ARMs	Да Для PixelCNN – 1 inference step	Да Для PixelCNN – $h * w$ inference steps
GANs	Нет Likelihood-free algorithm	Да Как правило генерируется сразу все изображение за один раз -- 1 inference step

VAE: Variational Autoencoder

Bayesian Framework

Байесовская логика — это способ **обновлять знания** о неизвестных параметрах модели, когда мы получаем новые данные. То есть:

было предположение (prior), пришли новые наблюдения (data) → получили обновлённое знание (posterior).

♦ 2. Формула Байеса

$$p(t | x) = \frac{p(x | t)p(t)}{p(x)} = \frac{p(x | t)p(t)}{\int p(x | t)p(t)dt}$$

где:

t — скрытые параметры (например, веса сети, вероятности классов, факторы);

x — наблюдаемые данные;

$p(x | t)$ — *likelihood* (насколько хорошо параметры объясняют данные);

$p(t)$ — *prior* (наше знание до наблюдения);

$p(x)$ — *evidence*, нормировочная константа;

$p(t | x)$ — *posterior*, обновлённое распределение после наблюдения данных.

Bayesian Framework (Model Parameters Case)



Теперь предположим, что **скрытые переменные $\mathbf{t}$** — это **параметры модели θ** .

$\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^n$ — *наблюдаемые данные*

$p(\theta)$ — *априорное распределение параметров (принимая θ как случайные переменные)*

Posterior distribution

$$p(\theta | \mathbf{X}) = \frac{p(\mathbf{X} | \theta) p(\theta)}{p(\mathbf{X})} = \frac{p(\mathbf{X} | \theta) p(\theta)}{\int p(\mathbf{X} | \theta) p(\theta) d\theta}$$

Interpretation

$p(\mathbf{X} | \theta)$ — **likelihood**: насколько хорошо параметры объясняют данные.

$p(\theta)$ — **prior**: наше знание о параметрах до обучения.

$p(\theta | \mathbf{X})$ — **posterior**: обновлённое знание после наблюдений.

$p(\mathbf{X})$ — **evidence**: нормирующий множитель (не зависит от θ).

Latent Variable Models (LVM)

Мы ищем параметры модели θ^* , которые **максимизируют вероятность наблюдаемых данных**:

$$\theta^* = \arg \max_{\theta} p(\mathbf{X} | \theta) = \arg \max_{\theta} \prod_{i=1}^n p(\mathbf{x}_i | \theta) = \arg \max_{\theta} \sum_{i=1}^n \log p(\mathbf{x}_i | \theta)$$

Проблема: распределение $p(\mathbf{x} | \theta)$ часто **сложно задать явно** и **невозможно посчитать напрямую** — особенно для реальных данных $\pi(\mathbf{x})$.

Extended Probabilistic Model

Чтобы сделать задачу tractable, вводим **латентные переменные $\mathbf{z}$** (скрытые факторы, объясняющие наблюдения)

$$\begin{aligned} p(\mathbf{x}, \mathbf{z} | \theta) &= p(\mathbf{x}, \mathbf{z} | \theta) p(\mathbf{z}) \\ \log p(\mathbf{x}, \mathbf{z} | \theta) &= \log p(\mathbf{x}, \mathbf{z} | \theta) + \log p(\mathbf{z}) \\ p(\mathbf{x} | \theta) &= \int p(\mathbf{x}, \mathbf{z} | \theta) d\mathbf{z} = \int p(\mathbf{x} | \mathbf{z}, \theta) p(\mathbf{z}) d\mathbf{z} \end{aligned}$$

Motivation

- 1) Хотя $p(\mathbf{x} | \theta)$ может быть очень сложным, распределения $p(\mathbf{x} | \mathbf{z}, \theta)$ и $p(\mathbf{z})$ часто можно сделать **простыми и интерпретируемыми**.
- 2) Латентная переменная $\mathbf{z}$ выступает как *скрытое представление* — компактное и структурированное описание данных.

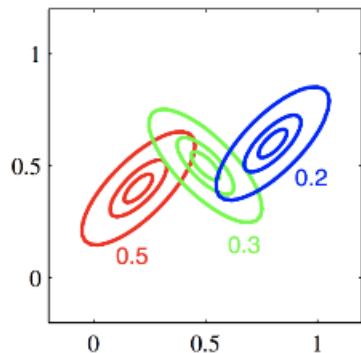
Ключевая идея

Вместо того чтобы напрямую моделировать сложное распределение $p(\mathbf{x} | \theta)$, мы описываем его через *простые распределения* над скрытыми переменными $\mathbf{z}$.

Latent Variable Models (LVM)

Examples

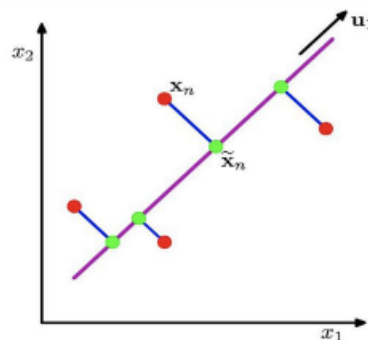
Mixture of gaussians



► $p(\mathbf{x}|\mathbf{z}, \theta) = \mathcal{N}(\mathbf{x}|\mu_{\mathbf{z}}, \Sigma_{\mathbf{z}})$

► $p(\mathbf{z}) = \text{Categorical}(\pi)$

PCA model



► $p(\mathbf{x}|\mathbf{z}, \theta) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \mu, \sigma^2\mathbf{I})$

► $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|0, \mathbf{I})$

Variational Lower Bound (ELBO)

Вывод ELBO 1

- (1) Распишем по определению
- (2) Умножим и разделим на $q(z)$
- (3) По определению **LOTUS**
- (4) Jensen's Inequality**
- (5) ELBO

$$\begin{aligned}\log p(\mathbf{x}|\theta) &\stackrel{(1)}{=} \log \int p(\mathbf{x}, \mathbf{z}|\theta) d\mathbf{z} \stackrel{(2)}{=} \log \int \frac{q(\mathbf{z})}{q(\mathbf{z})} p(\mathbf{x}, \mathbf{z}|\theta) d\mathbf{z} \stackrel{(3)}{=} \\ &= \log \mathbb{E}_q \left[\frac{p(\mathbf{x}, \mathbf{z}|\theta)}{q(\mathbf{z})} \right] \stackrel{(4)}{\geq} \mathbb{E}_q \log \frac{p(\mathbf{x}, \mathbf{z}|\theta)}{q(\mathbf{z})} \stackrel{(5)}{=} \mathcal{L}(q, \theta)\end{aligned}$$

Вывод ELBO 2

- (1) Распишем по определению (**Jensen's Inequality**)
- (2) Расписали совместное распределение
- (3) По правилу умножения в логарифме
- (4) Левый множитель с логарифмом выносим за интеграл, потому что от z не зависит, а то что осталось в интеграле рано 1 *Because $\int q(z|\mathbf{x}) dz = 1$*
- (5) Правый множитель это KL по определению

$$\begin{aligned}\mathcal{L}(q, \theta) &\stackrel{(1)}{=} \int q(\mathbf{z}) \log \frac{p(\mathbf{x}, \mathbf{z}|\theta)}{q(\mathbf{z})} d\mathbf{z} \stackrel{(2)}{=} \int q(\mathbf{z}) \log \frac{p(\mathbf{z}|\mathbf{x}, \theta) p(\mathbf{x}|\theta)}{q(\mathbf{z})} d\mathbf{z} \stackrel{(3)}{=} \\ &= \int q(\mathbf{z}) \log p(\mathbf{x}|\theta) d\mathbf{z} + \int q(\mathbf{z}) \log \frac{p(\mathbf{z}|\mathbf{x}, \theta)}{q(\mathbf{z})} d\mathbf{z} = \\ &= \log p(\mathbf{x}|\theta) - KL(q(\mathbf{z}) || p(\mathbf{z}|\mathbf{x}, \theta))\end{aligned}$$

Связь между Likelihood и ELBO: $\log p(\mathbf{x}|\theta) = \mathcal{L}(q, \theta) + KL(q(\mathbf{z}) || p(\mathbf{z}|\mathbf{x}, \theta)) \geq \mathcal{L}(q, \theta).$

Variational Lower Bound (ELBO)

Вывод ELBO 3

- (1) Расписали совместное распределение
- (2) По определению **LOTUS**
- (3) KL

$$\begin{aligned}\mathcal{L}(q, \theta) &= \int q(\mathbf{z}) \log \frac{p(\mathbf{x}, \mathbf{z} | \theta)}{q(\mathbf{z})} d\mathbf{z} \stackrel{(1)}{=} \\ &= \int q(\mathbf{z}) \log p(\mathbf{x} | \mathbf{z}, \theta) d\mathbf{z} + \int q(\mathbf{z}) \log \frac{p(\mathbf{z})}{q(\mathbf{z})} d\mathbf{z} \\ &= \underbrace{\mathbb{E}_q \log p(\mathbf{x} | \mathbf{z}, \theta)}_{(2)} - \underbrace{KL(q(\mathbf{z}) || p(\mathbf{z}))}_{(3)}\end{aligned}$$

Связь между Likelihood и ELBO:

$$\begin{aligned}\log p(\mathbf{x} | \theta) &= \mathcal{L}(q, \theta) + KL(q(\mathbf{z}) || p(\mathbf{z} | \mathbf{x}, \theta)) \\ &= \mathbb{E}_q \log p(\mathbf{x} | \mathbf{z}, \theta) - KL(q(\mathbf{z}) || p(\mathbf{z})) + KL(q(\mathbf{z}) || p(\mathbf{z} | \mathbf{x}, \theta)).\end{aligned}$$

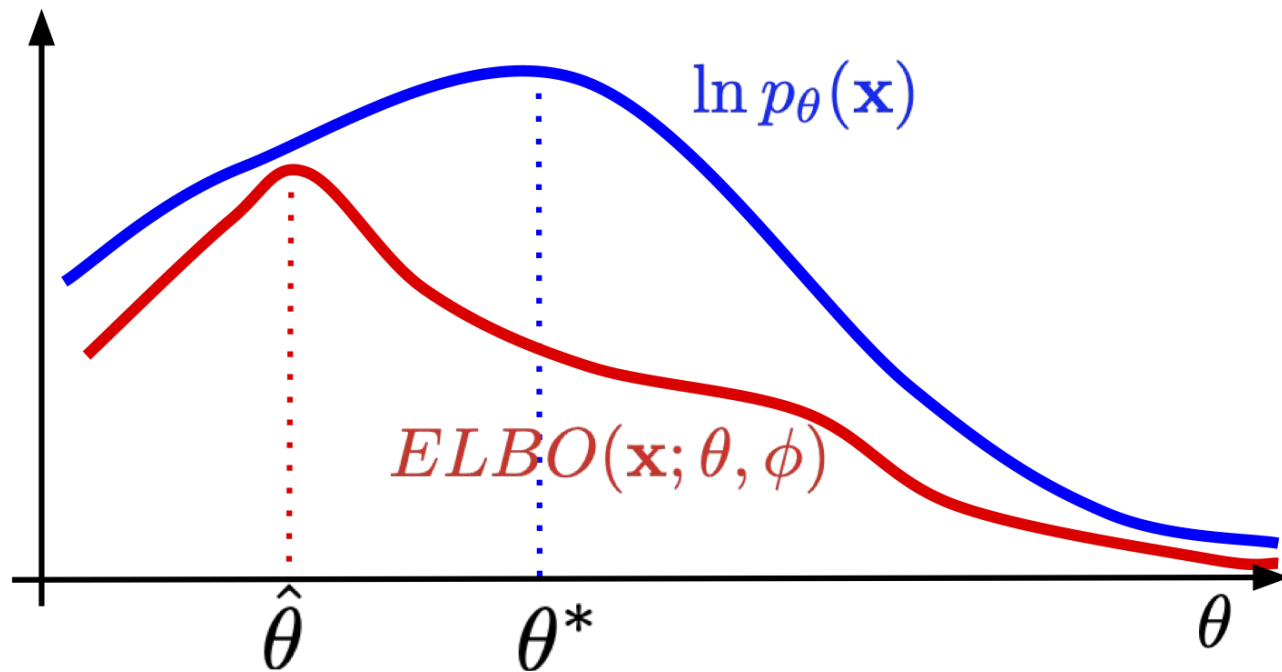
- Instead of maximizing incomplete likelihood, maximize ELBO

$$\max_{\theta} p(\mathbf{x} | \theta) \rightarrow \max_{q, \theta} \mathcal{L}(q, \theta)$$

- Maximization of ELBO by **variational** distribution q is equivalent to minimization of KL

$$\arg \max_q \mathcal{L}(q, \theta) \equiv \arg \min_q KL(q(\mathbf{z}) || p(\mathbf{z} | \mathbf{x}, \theta)).$$

Связь между Likelihood и ELBO



Построим VAE

Loss function: $\mathcal{L}(q, \theta) = \mathbb{E}_q \log p(\mathbf{x}|\mathbf{z}, \theta) + KL(q(\mathbf{z})||p(\mathbf{z}|\mathbf{x}, \theta)).$

$\mathbb{E}_q \log p(\mathbf{x}|\mathbf{z}, \theta)$

- $P(\mathbf{x} | \mathbf{z}, \theta)$ -- Нейросеть генератор, которая принимает на вход $\mathbf{z}$ и генерирует $\mathbf{x}$
- Часто во время обучения заменяется на $MSE(\mathbf{x}_{true}, \mathbf{x}_{pred})$

$KL(q(\mathbf{z})||p(\mathbf{z}|\mathbf{x}, \theta))$

- Чаще всего $q(\mathbf{z}) = N(0; I)$
- И тогда $p(\mathbf{z} | \mathbf{x}, \theta)$ – другая нейросеть, которая представляется в следующем виде $\mathcal{N}(\mathbf{z}|\mu_\phi(\mathbf{x}), \sigma_\phi^2(\mathbf{x}))$;
- Тогда KL просто рассчитывается просто по формуле между двумя нормальными распределениями

Вопрос:

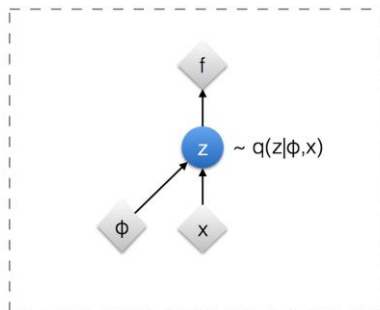
Как нам тогда во время обучения получать $\mathbf{z}$ – его же нужно сэмплировать. А если просэмплировать, то как считать градиенты?

$$\begin{aligned} KL[P||Q] &= \frac{1}{2} \left[-\frac{\sigma_1^2}{\sigma_2^2} + \frac{\mu_1^2 + \sigma_1^2 - 2\mu_2\mu_1 + \mu_2^2}{\sigma_2^2} - \ln \frac{\sigma_1^2}{\sigma_2^2} \right] \\ &= \frac{1}{2} \left[\frac{\mu_1^2 - 2\mu_1\mu_2 + \mu_2^2}{\sigma_2^2} + \frac{\sigma_1^2}{\sigma_2^2} - \ln \frac{\sigma_1^2}{\sigma_2^2} - 1 \right] \\ &= \frac{1}{2} \left[\frac{(\mu_1 - \mu_2)^2}{\sigma_2^2} + \frac{\sigma_1^2}{\sigma_2^2} - \ln \frac{\sigma_1^2}{\sigma_2^2} - 1 \right] \end{aligned}$$

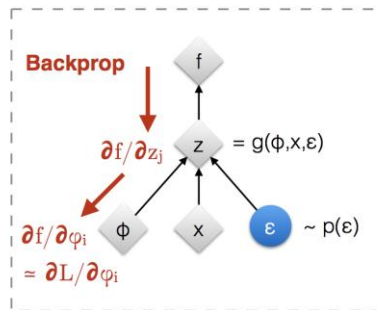
Reparameterization Trick

$$\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)} \mathbf{I})$$
$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad ; \text{ Reparameterization trick.}$$

Original form



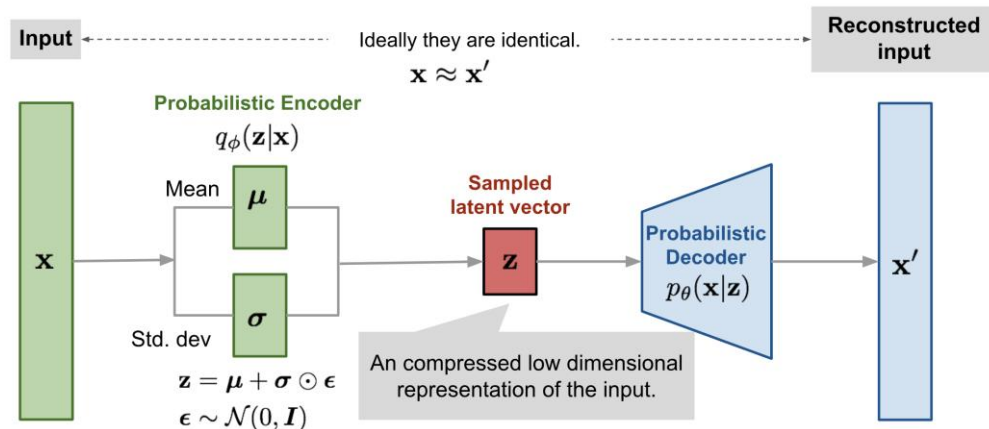
Reparameterised form



 : Deterministic node
 : Random node

[Kingma, 2013]
[Bengio, 2013]
[Kingma and Welling 2014]
[Rezende et al 2014]

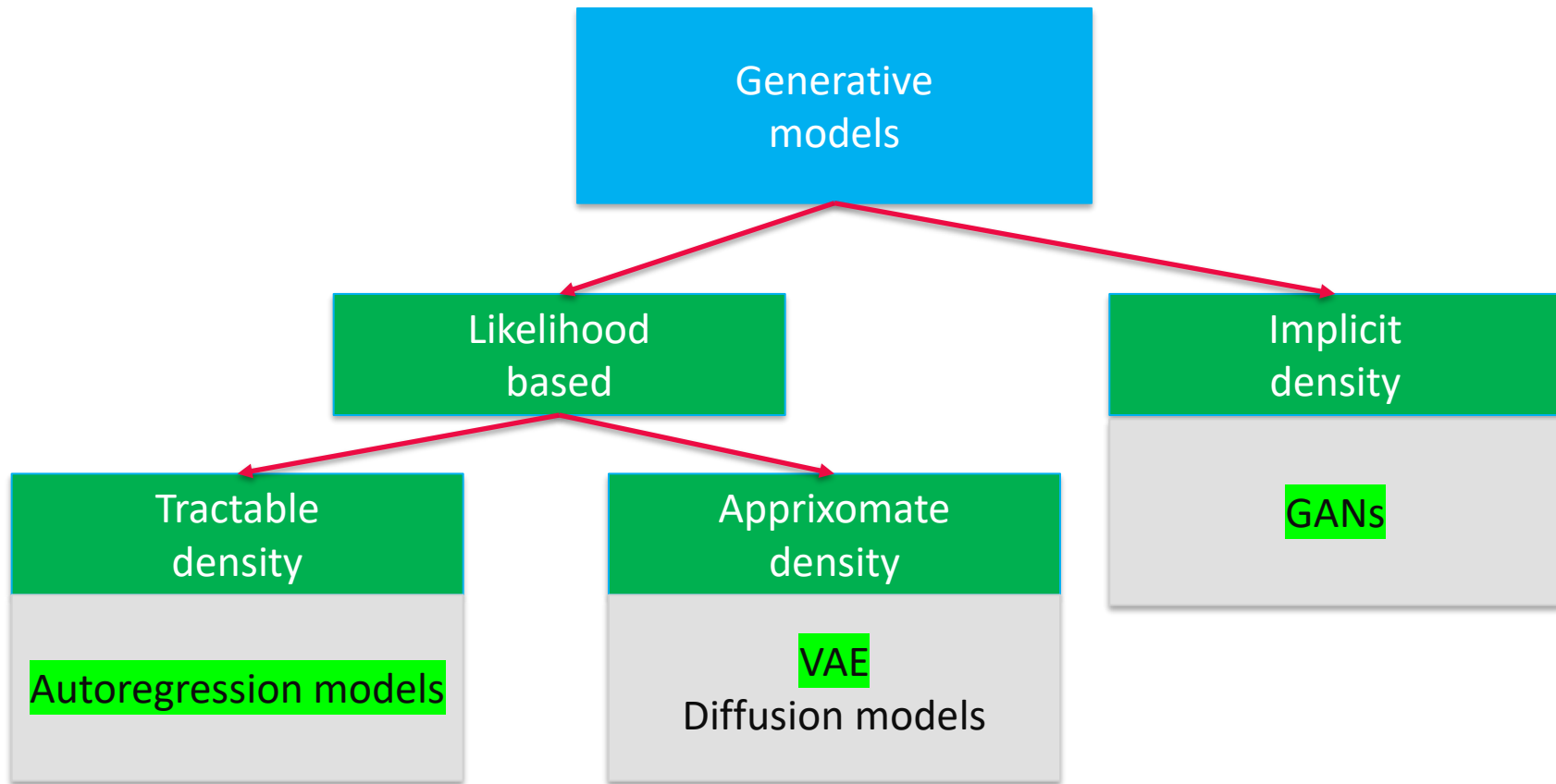
VAE train



So as you can see, the whole math boils down to a relatively simple learning procedure:

1. Take $\mathbf{x}_n$ and apply the encoder network to get $\mu_\phi(\mathbf{x}_n)$ and $\ln \sigma_\phi^2(\mathbf{x}_n)$.
2. Calculate $\mathbf{z}_{\phi,n}$ by applying the reparameterization trick, $\mathbf{z}_{\phi,n} = \mu_\phi(\mathbf{x}_n) + \sigma_\phi(\mathbf{x}_n) \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.
3. Apply the decoder network to $\mathbf{z}_{\phi,n}$ to get the probabilities $\theta(\mathbf{z}_{\phi,n})$.
4. Calculate the ELBO by plugging in $\mathbf{x}_n$, $\mathbf{z}_{\phi,n}$, $\mu_\phi(\mathbf{x}_n)$ and $\ln \sigma_\phi^2(\mathbf{x}_n)$.

Зоопарк генеративных моделей



**Спасибо
за внимание!**

it'sMO *re than a*
UNIVERSITY